



CS-4110 HPC with GPUs

Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E



Today's lecture outline

- Historical note and background
 - GPU System Architectures
 - Programming GPUs
 - Multithreaded Multiprocessor Architecture
 - Practical demonstrations
-

Historical note and background

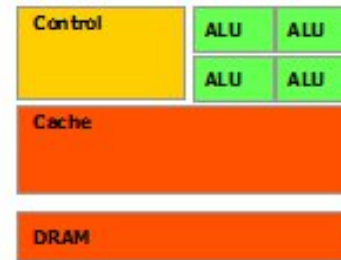
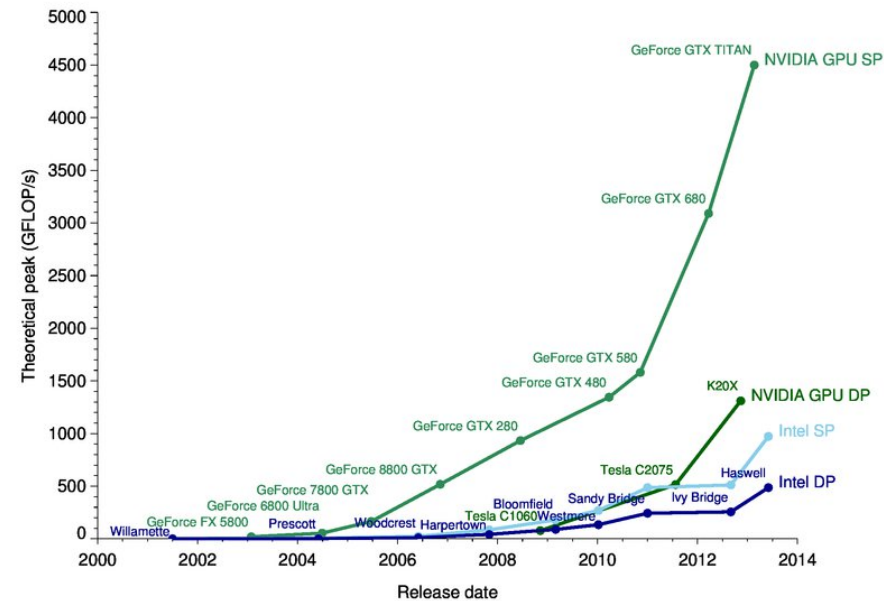
- 1980s video graphics array (VGA) controllers did computer graphics
- 1990s complex 3D rendering
 - triangle setup
 - texture mapping
 - shading
- 2000s graphics processing unit (GPU)
 - graphics device become processor
- 2010s general-purpose GPU (GPGPU)
 - GPU programmable as CPU



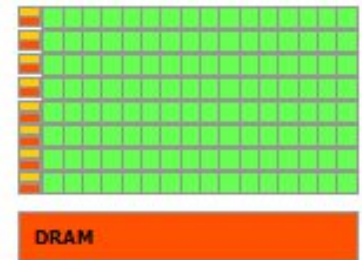
VGA graphics

Historical note and background

- GPU peak performance much higher than CPU
- Only achievable for highly parallel applications
 - Graphics
 - Scientific
 - Etc.
- Made possible by many small GPU cores



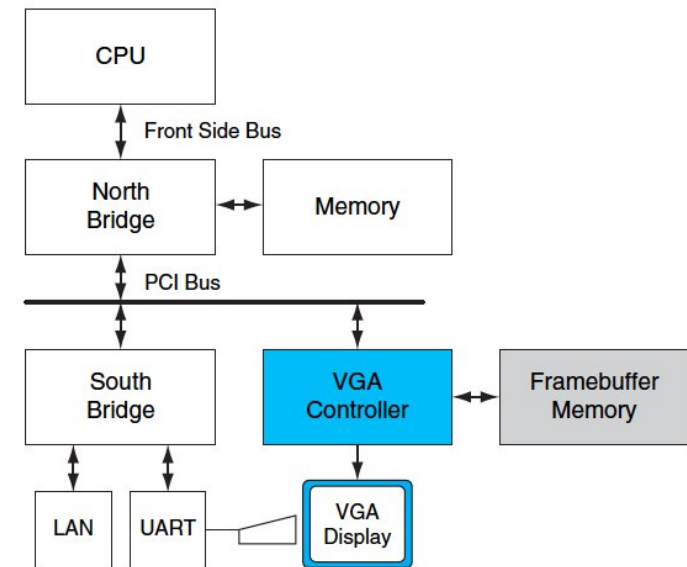
CPU



GPU

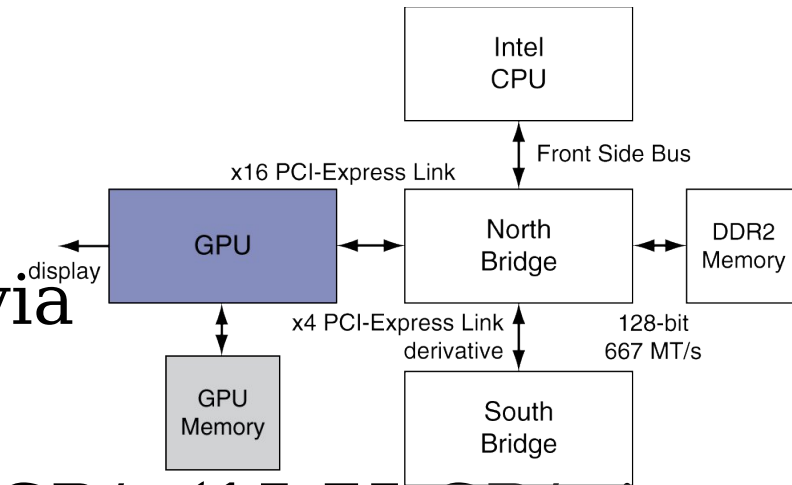
GPU System Architectures

- Systems using GPUs and CPUs are heterogeneous
- Such systems can be classified by 2 characteristics
 - Memory subsystem
 - The interconnect
- Historical PC (circa 1990)
 - North bridge for high-bandwidth interfaces
 - South bridge for legacy interfaces



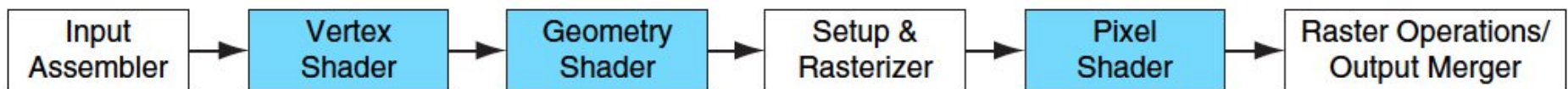
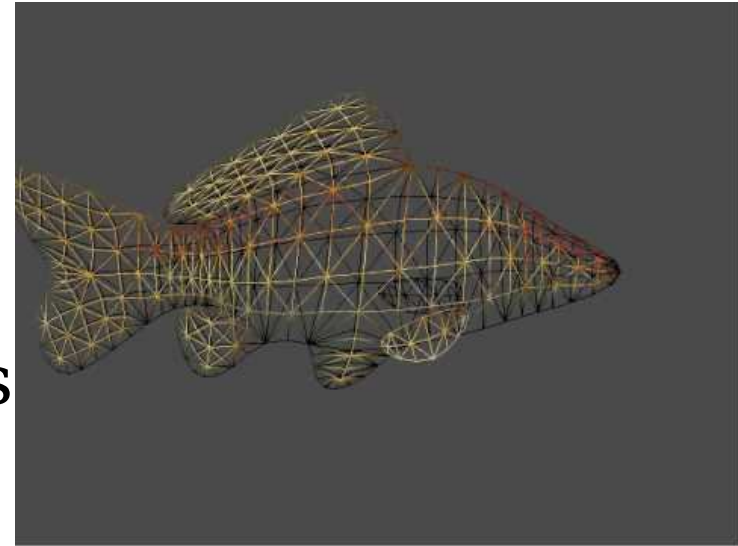
GPU System Architectures

- Modern architecture
 - separate CPU & GPU
 - Each has own memory
 - PCIe x16 interconnect
- High speed interconnect via north bridge
- PCIe x16 v3 enables 31.5 GB/s (15.75 GB/s in each direction)
- Low cost systems use only CPU system memory, omitting GPU memory



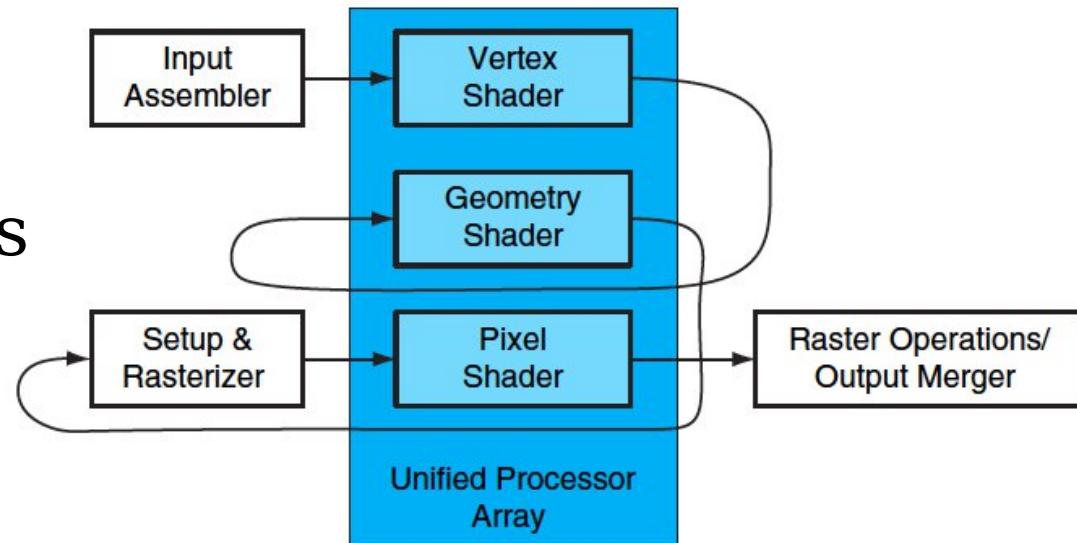
GPU System Architectures

- All images are made of tiny triangles
- Graphics processing requires 6 steps
- 3 steps are programmable
 - Vertex shader
 - Geometry shader
 - Pixel shader
- Rest are fixed function steps

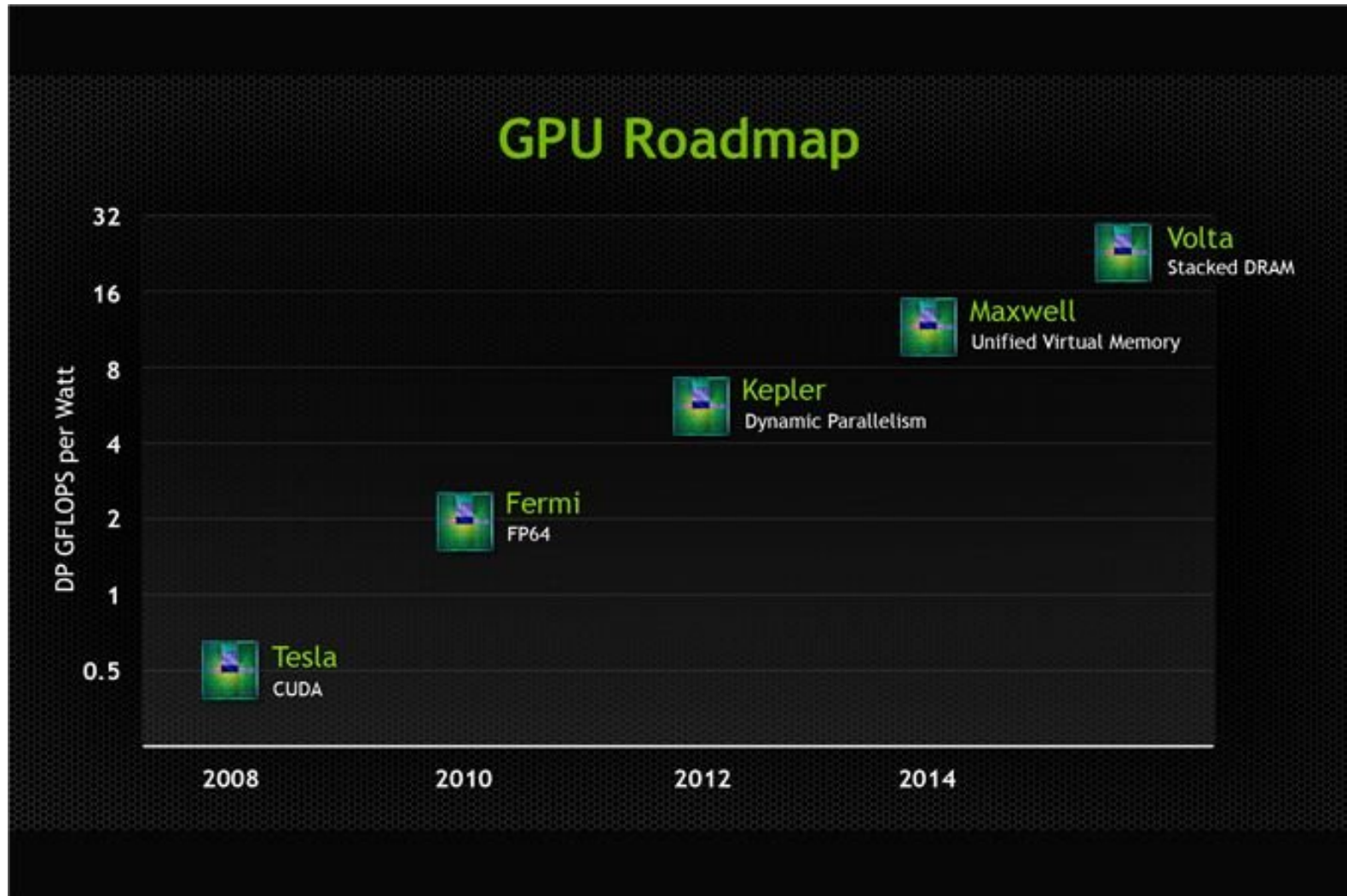


GPU System Architectures

- Modern GPU combine the programmable steps into one single processor unit (PU)
 - Processing is accelerated by duplicating PUs in a Unified Processor Array
-
- The diagram illustrates the data flow in a GPU. It features a central blue vertical bar containing three light blue boxes: 'Vertex Shader', 'Geometry Shader', and 'Pixel Shader'. To the left of this bar, an 'Input Assembler' box has an arrow pointing to the 'Vertex Shader'. Below it, a 'Setup &' box has an arrow pointing to the 'Geometry Shader'. To the right of the bar, a 'Raster Opera' box has an arrow pointing from the 'Pixel Shader'. A curved arrow loops from the 'Vertex Shader' to the 'Geometry Shader', and another curved arrow loops from the 'Geometry Shader' to the 'Pixel Shader', indicating a sequential processing flow. Additionally, a curved arrow loops from the 'Vertex Shader' back to the 'Input Assembler'.
- GPU can process many more threads than CPUs efficiently



GPU System Architectures

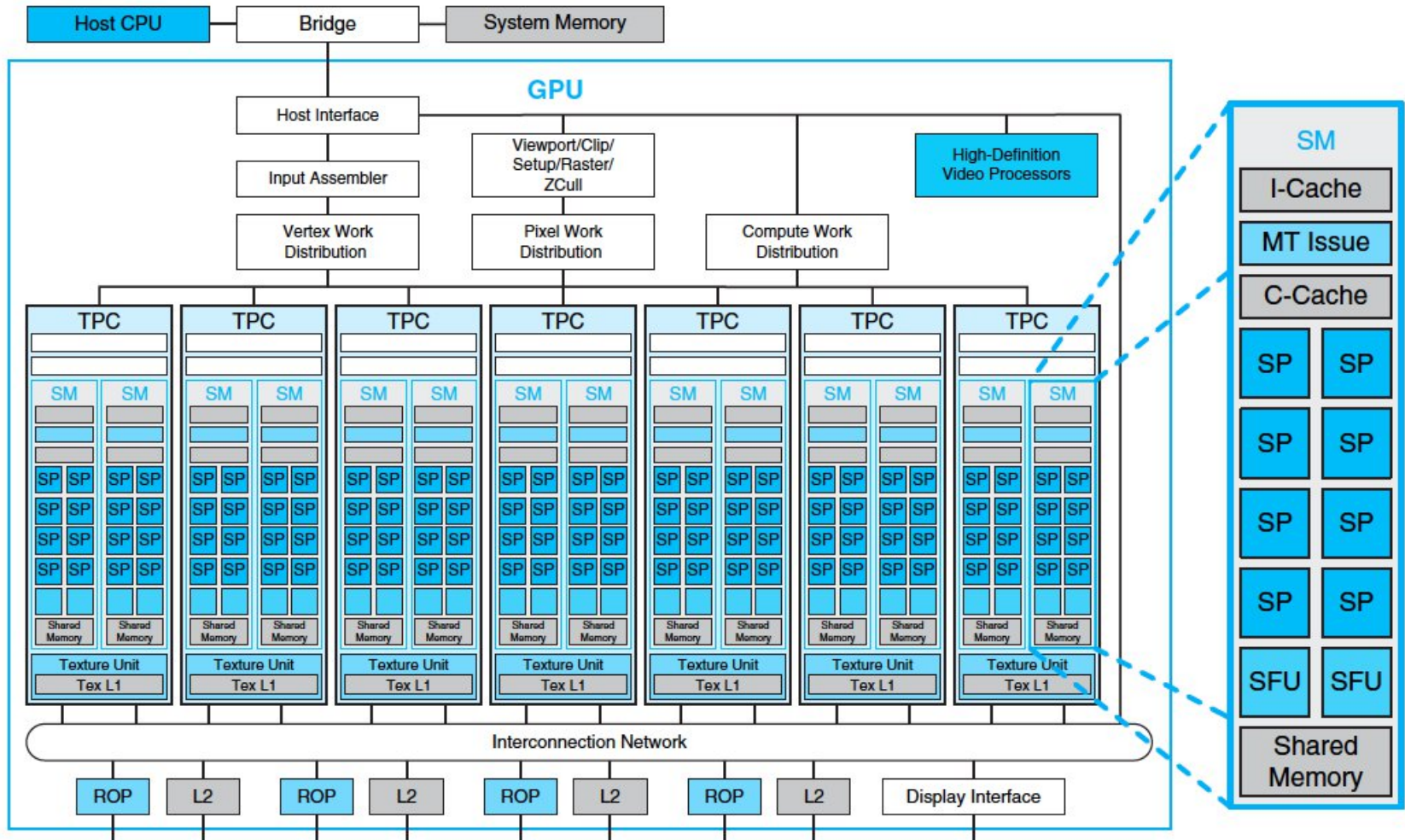


GPU System Architectures

GPT-4 1.8T MoE	2016	2018	2020	2022	2024
10 Day Training	"Pascal"	"Volta"	"Ampere"	"Hopper"	"Blackwell"
GPU	P100	V100	A100	H100	B100
Peak Teraflops	19	130	620	4,000	20,000
<i>Training And Inference Precision</i>	<i><u>FP16</u></i>	<i><u>FP16</u></i>	<i><u>FP16</u></i>	<i><u>FP8</u></i>	<i><u>FP4</u></i>
Inference Joules/Token	17,000	1,200	150	10	0.4
GPUs to Train In 10 Days	42,105,300	6,153,800	1,290,300	100,000	10,000
GPU Unit Price	\$5,000	\$10,000	\$15,000	\$27,500	\$37,500
<i>GPU Cost For Training In 10 Days</i>	<i><u>\$210.5 B</u></i>	<i><u>\$61.5 B</u></i>	<i><u>\$19.4 B</u></i>	<i><u>\$2.8 B</u></i>	<i><u>\$0.4 B</u></i>
Power To Train, Gw-hour	1,000	140	40	13	3
Electricity Cost Per Kw-hour	\$0.14	\$0.14	\$0.14	\$0.16	\$0.18
<i>Electricity Cost For Training Run</i>	<i><u>\$140.0 M</u></i>	<i><u>\$19.6 M</u></i>	<i><u>\$5.6 M</u></i>	<i><u>\$2.1 M</u></i>	<i><u>\$0.54 M</u></i>
<i>Three Year Electricity Cost</i>	<i><u>\$15.3 B</u></i>	<i><u>\$2.1 B</u></i>	<i><u>\$613.6 M</u></i>	<i><u>\$227.9 M</u></i>	<i><u>\$59.2 M</u></i>

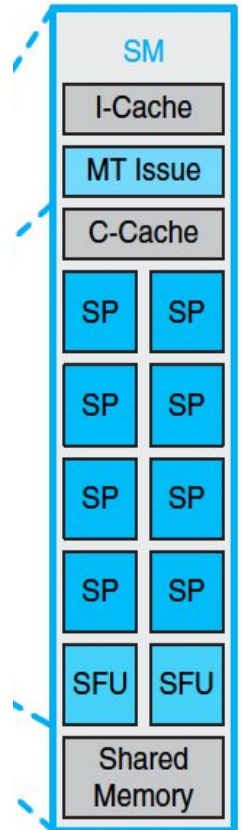
GPU System Architectures: Tesla architecture

- The unified processor array of NVIDIA



GPU System Architectures: Tesla architecture

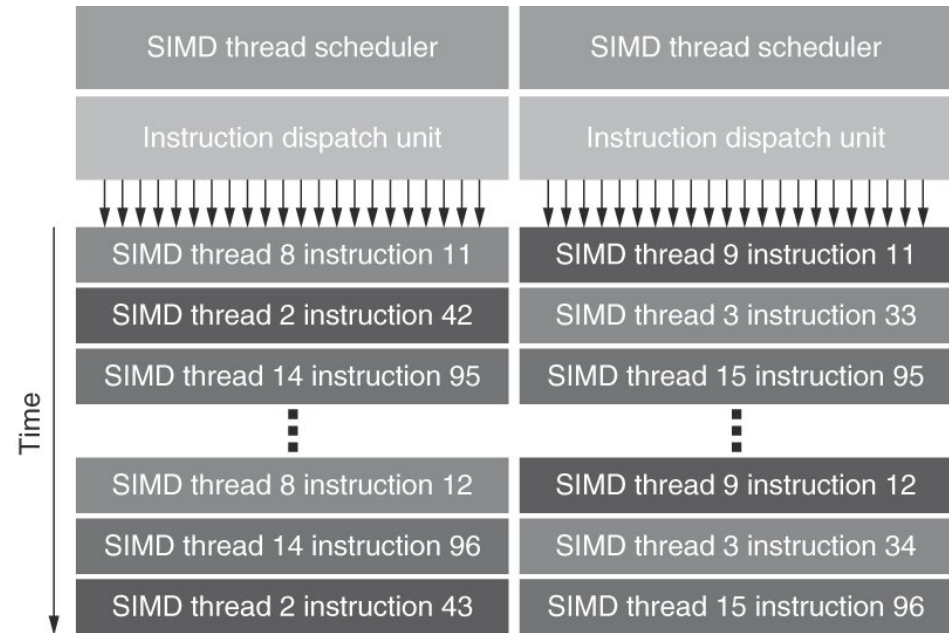
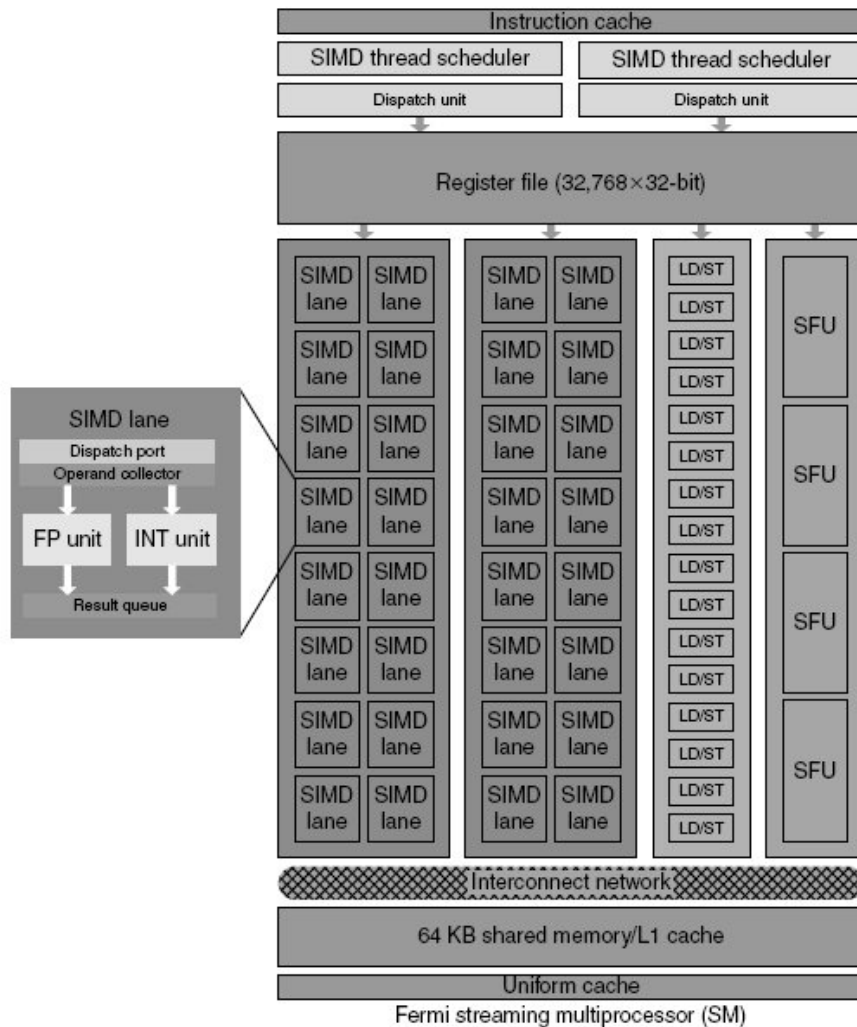
- Tesla architecture in NVIDIA GeForce 8800
- Array of 112 streaming processor (SP) cores
- 14 multithreaded streaming multiprocessors (SM)
- SP core is highly multithreaded, managing 96 concurrent threads
- Each SM has
 - 8 SP cores
 - 2 special function units (SFUs)
 - instruction and constant caches
 - multithreaded instruction unit
 - shared memory



Compute Capability

- The compute capability is the "feature set" (both hardware and software features) of the device
- GPU architectures have features that previous versions might not have
- What is the compute capability of the GPU in your machine?
- `nvidia-smi --query-gpu=compute_cap --format=csv`

GPU System Architectures: Fermi architecture

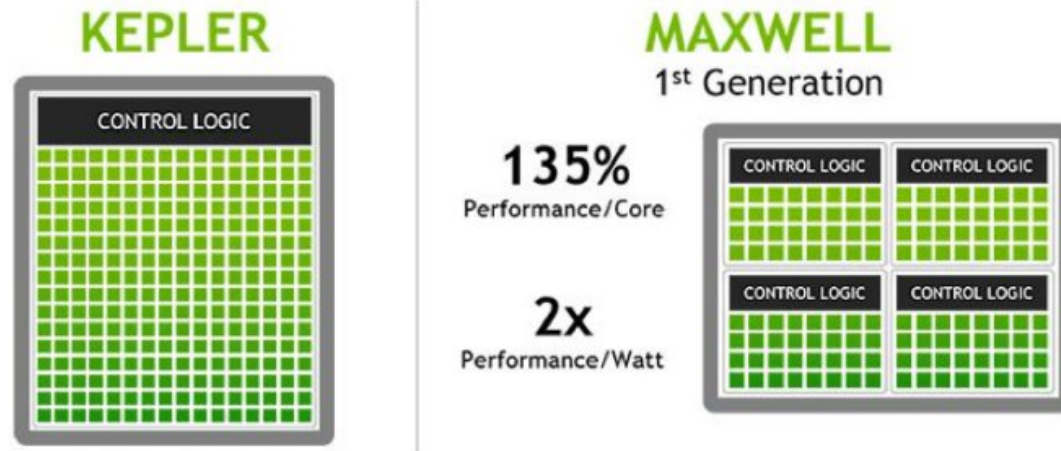


GPU System Architectures: Fermi architecture

- Each SIMD processor has
 - 2 SIMD thread schedulers, 2 instruction dispatch units
 - 16 SIMD lanes, 16 load-store units, 4 SFUs
 - Thus, 2 SIMD threads scheduled every 2 clock cycles
 - Fast double precision: 515 GFLOPs for DAXPY
 - GPU memory caches : I/D L1/SIMD proc & shared L2
 - 64-bit addressing & unified address space: C/C++ ptrs
 - Faster context switching: hardware support, 10x faster
 - Error correcting codes: dependability for long-running apps
-

GPU System Architectures: Maxwell arch.

- Maxwell divides SPs into smaller blocks
- Each block has extra control logic
- Allows switching on/off SPs at a smaller granularity
- Results in higher energy efficiency



CUDA vs Tensor Cores

- CUDA Cores
 - designed to perform a wide range of floating-point and integer operations
- Tensor Cores
 - can perform operations like 4x4 matrix multiply-accumulate (GEMM) in a single instruction, providing a performance boost for DL training and inference.
- NVIDIA GeForce RTX 3080
 - 8704 CUDA cores
 - 272 Tensor cores

CUDA vs Tensor Cores

- CUDA cores are general-purpose processing units, while Tensor cores are specialized for DL matrix operations.
- The number of CUDA cores in a GPU provides a measure of the overall computational power, while the number of Tensor cores indicates the DL performance.
- Modern NVIDIA GPUs like the H100 contain both CUDA cores and Tensor cores to provide a balance of general-purpose and DL-specific acceleration.

Programming GPUs

- Historically: real-time graphics performance needed
 - complex 3D scenes
 - high resolution
 - high frame rate (60 frames per second)
- Scalable programming models used
 - graphics shading languages, Cg (C for graphics)
 - HLSL (high-level shading language)
- Exploit large degree of parallelism
 - many independent parallel threads
 - scale to any number of SPs

Programming GPUs

- Compute Unified Device Architecture (CUDA)
 - scalable parallel programming model
 - software platform for parallel processors
- Allows programmer to bypass the graphics API
 - simply program in C or C++
- CUDA model is SPMD (single-program multiple data)
 - program written for one thread
 - duplicated by CUDA multiple times
 - executed in parallel on available SPs
- CUDA can program multiple CPU cores as well
 - programs the entire heterogeneous computer system

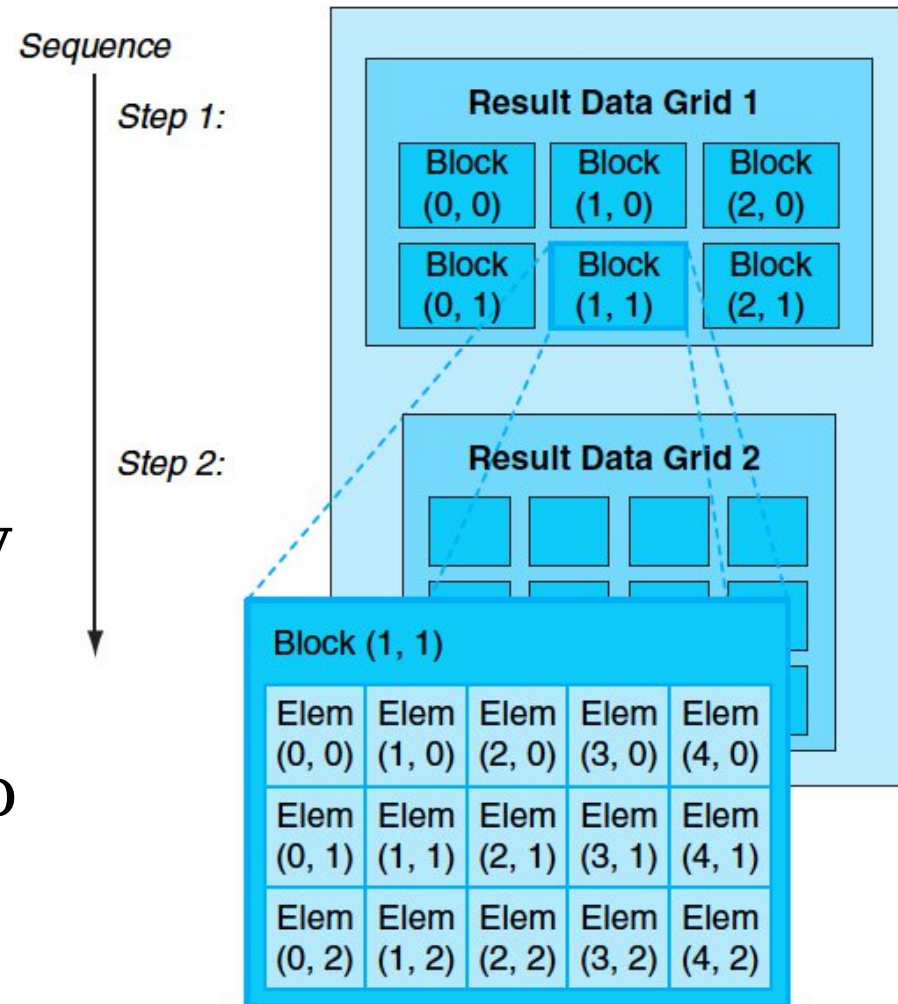
CUDA support

Supported CUDA level of GPU and card. See also at [Nvidia](#):

- CUDA SDK 1.0 support for compute capability 1.0 – 1.1 (Tesla)^[29]
- CUDA SDK 1.1 support for compute capability 1.0 – 1.1+x (Tesla)
- CUDA SDK 2.0 support for compute capability 1.0 – 1.1+x (Tesla)
- CUDA SDK 2.1 – 2.3.1 support for compute capability 1.0 – 1.3 (Tesla)^{[30][31][32][33]}
- CUDA SDK 3.0 – 3.1 support for compute capability 1.0 – 2.0 (Tesla, Fermi)^{[34][35]}
- CUDA SDK 3.2 support for compute capability 1.0 – 2.1 (Tesla, Fermi)^[36]
- CUDA SDK 4.0 – 4.2 support for compute capability 1.0 – 2.1+x (Tesla, Fermi, more?).
- CUDA SDK 5.0 – 5.5 support for compute capability 1.0 – 3.5 (Tesla, Fermi, Kepler).
- CUDA SDK 6.0 support for compute capability 1.0 – 3.5 (Tesla, Fermi, Kepler).
- CUDA SDK 6.5 support for compute capability 1.1 – 5.x (Tesla, Fermi, Kepler, Maxwell). Last version with support for compute capability 1.0.
- CUDA SDK 7.0 – 7.5 support for compute capability 2.0 – 5.x (Fermi, Kepler, Maxwell).
- CUDA SDK 8.0 support for compute capability 2.0 – 6.x (Fermi, Kepler, Maxwell, Pascal). Last version with support for compute capability 1.0.
- CUDA SDK 9.0 – 9.2 support for compute capability 3.0 – 7.2 (Kepler, Maxwell, Pascal, Volta) (Pascal GTX 1070Ti Not Supported).
- CUDA SDK 10.0 – 10.2 support for compute capability 3.0 – 7.5 (Kepler, Maxwell, Pascal, Volta, Turing). Last version with support for macOS, as support will not be available for macOS in newer releases.
- CUDA SDK 11.0 support for compute capability 3.5 – 8.0 (Kepler (in part), Maxwell, Pascal, Volta, Turing, Ampere (in part)).^[37] Cores.^[38]
- CUDA SDK 11.1 – 11.3 support for compute capability 3.5 – 8.6 (Kepler (in part), Maxwell, Pascal, Volta, Turing, Ampere).^[39]

Programming GPUs

- Problem partition needed for efficient GPU utilization
- Small problems can be solved in parallel
- Large result data array divided into blocks
 - Computed by SPs
- Each block divided into elements
 - Computed by threads



Programming GPUs

- Programmer defines kernels to process data in grids
- Grid is a set of thread blocks
 - each executed independently
 - may execute in parallel
- Thread block is a set of concurrent threads
 - cooperate through barrier synchronization
- Programmer specifies number of threads per block and number of blocks in the grid
 - threadIdx is thread ID in block
 - blockIdx is a unique block ID number

Programming GPUs

- Example to calculate SAXPY
 - compute the result of $y = ax + y$

Computing $y = ax + y$ with a serial loop:

```
void saxpy_serial(int n, float alpha, float *x, float *y)
{
    for(int i = 0; i < n; ++i)
        y[i] = alpha*x[i] + y[i];
}

// Invoke serial SAXPY kernel
saxpy_serial(n, 2.0, x, y);
```

Programming GPUs

- Example to calculate SAXPY in CUDA

Computing $y = ax + y$ in parallel using CUDA:

```
__global__  
void saxpy_parallel(int n, float alpha, float *x, float *y)  
{  
    int i = blockIdx.x*blockDim.x + threadIdx.x;  
    if( i < n ) y[i] = alpha*x[i] + y[i];  
}  
  
// Invoke parallel SAXPY kernel (256 threads per block)  
int nblocks = (n + 255) / 256;  
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);
```

Programming GPUs

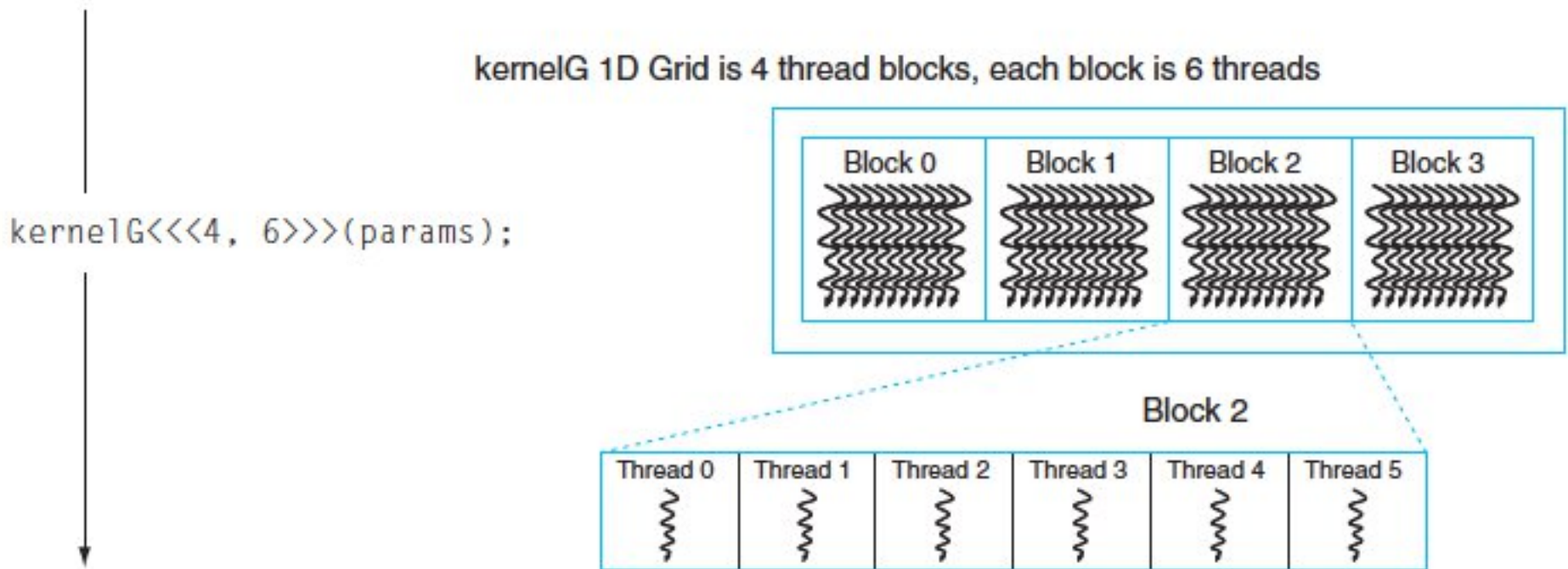
- `__global__` indicates CUDA kernel definition
- Kernels are called using

```
kernel<<<dimGrid, dimBlock>>>(... parameter list ...);
```

- `dimGrid` specifies the dimensions of the grid in blocks
- `dimBlock` specifies the dimensions of the blocks in threads

Programming GPUs

- CUDA style is single program multiple data (SPMD)
- But dynamically selects the right number of

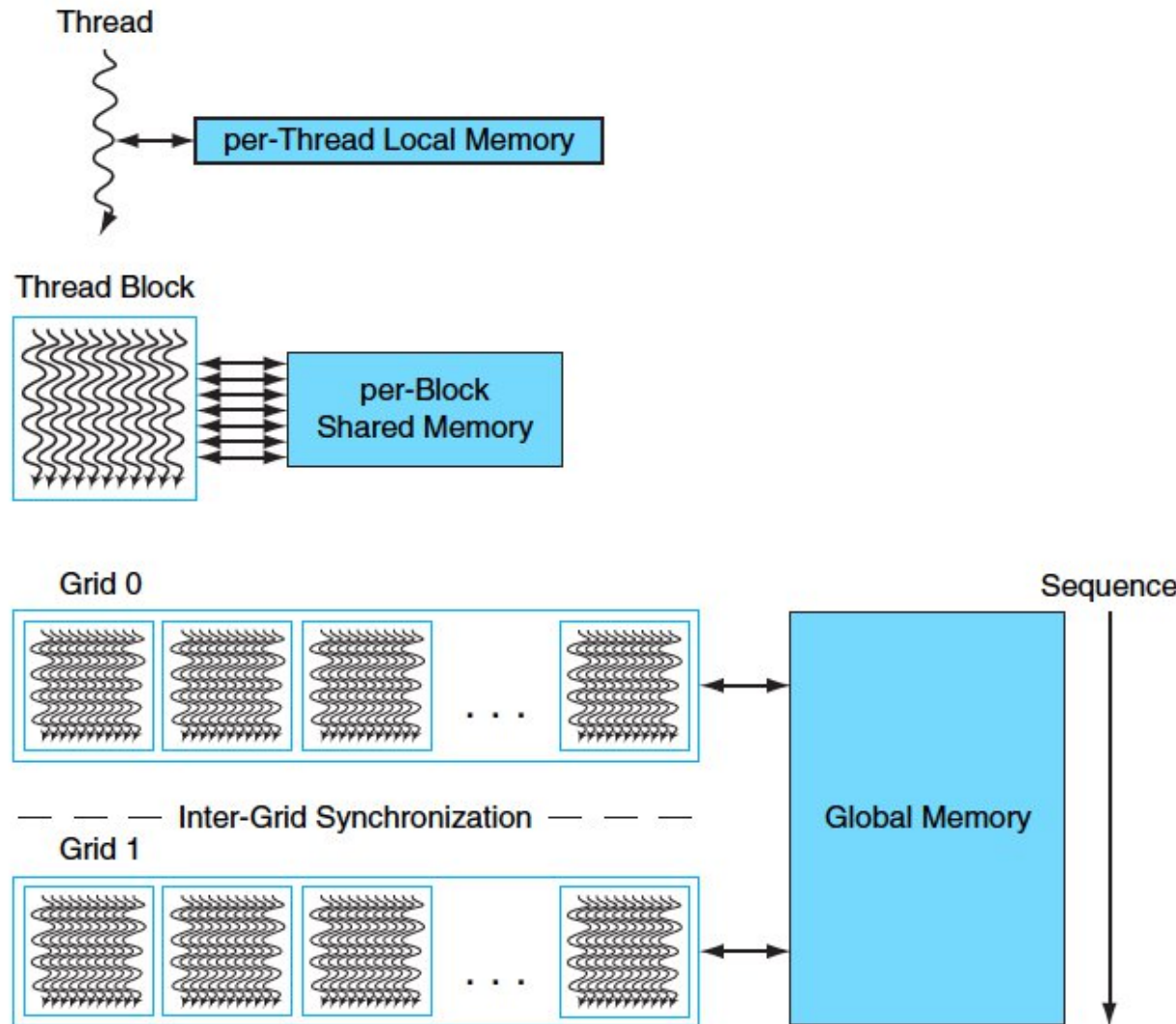


Programming GPUs

- The GPU has three levels of memory
- Local memory: Per-thread local memory private to the thread
- Shared memory: Per-block memory shared by all threads of the block
- Global memory: Per-application memory shared by all threads

Programming GPUs

- Shared memory low-latency memory near each SP similar to L1 cache



Multithreaded Multiprocessor Architecture

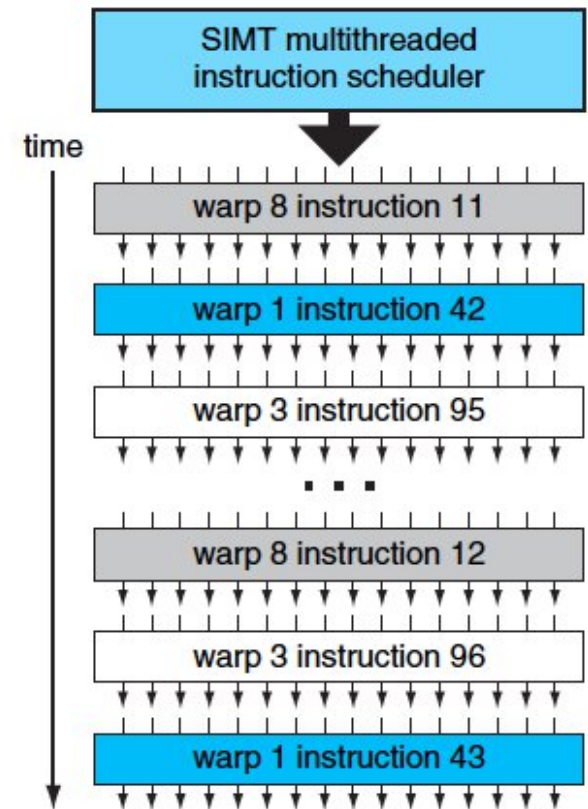
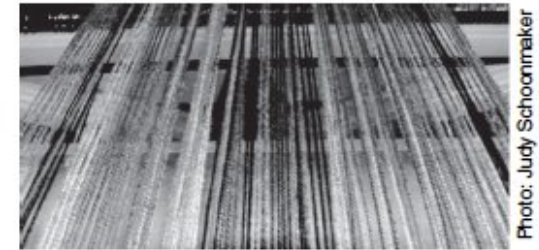
- GPUs implement scalable numbers of multiprocessors
- GPUs are multiprocessors composed of multiprocessors!!
- Each multiprocessor is highly multithreaded to execute many fine-grained vertex and pixel shader threads efficiently

Multithreaded Multiprocessor Architecture

- It can manage hundreds of threads
- Employs single-instruction multiple-thread (SIMT)
- Executes concurrent threads in groups of parallel threads called warps
- Warp originates from weaving, the first parallel thread technology!!

Multithreaded Multiprocessor Architecture

- Example of warp SIMT
- SIMT warp size of 32 threads
- Executing four threads in each of the eight SP cores over four clocks
- Tesla Architecture has warp size of 32 executing four threads per SP core



Multithreaded Multiprocessor Architecture

- Multithreading has to insure efficient inter-thread communication
- Command “__syncthreads()” allows threads to communicate frequently via shared memory
- It is possible to perform global synchronization for all threads
- It is also possible to perform warp-level synchronization (much more efficient)

Multithreaded Multiprocessor Architecture

- There are many levels for GPU memory
- Global memory is stored in external DRAM
- Shared memory is SRAM shared by threads in SM
- Local memory is private memory of a single thread
- Texture memory is shared by all threads and holds large read-only arrays of data
- Constant memory is read-only to threads on SM

Multithreaded Multiprocessor Architecture

- Efficient utilization of hardware is key!
- Minimize communication on GPU $\Leftrightarrow$ CPU
- Minimize GPU data transfers DRAM $\Leftrightarrow$ registers
- Minimize number registers to use all threads
- Minimize bank conflicts from in shared memory
- Maximize use of shared memory
- Maximize number of blocks running concurrently
- Conflicting requirements!

SIMD vs SIMT

- SIMD: Single Instruction, Multiple Data
- SIMT: Single Instruction, Multiple Threads
- Execution Model:
 - SIMD: Single instruction operates on multiple data elements simultaneously, typically using vector registers (e.g., in CPUs).
 - SIMT: Single instruction is executed by multiple threads BUT independently, allowing for divergence in execution (e.g., in GPUs).
- Control Flow:
 - SIMD: All data elements follow the same execution path without divergence.
 - SIMT: Threads can follow different execution paths, but divergence reduces efficiency.

SIMD vs SIMT

- Hardware Implementation:
 - SIMD: Found in CPUs (e.g., AVX, SSE) and some DSPs.
 - SIMT: Found in GPUs (e.g., NVIDIA CUDA, AMD ROCm).
- Memory Access:
 - SIMD: Operates on contiguous memory elements in vector registers.
 - SIMT: Uses independent memory accesses for each thread, with coalescing for efficiency.
- **SIMD is a strict parallel data processing model used in CPUs, while SIMT is a more flexible threading model used in GPUs.**

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- What are the kernel launch parameters in this example?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- What are the kernel launch parameters in this example?
 - 4 blocks in the grid
 - 128 threads per block

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many total threads are launched in this kernel execution?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- What is a warp in GPU execution?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- What is a warp in GPU execution?
- A warp is a group of 32 threads that execute together in lockstep.

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many warps are in each block?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many warps are in each block?
- Warps per Block=Threads per Block / Warp Size
$$=128 / 32 = 4$$

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many total warps are in the entire grid?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many total warps are in the entire grid?
- Total Warps=Blocks×Warps per Block= $4 \times 4 = 16$

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- If we have 2 SMs, how are the blocks distributed across them?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- If we have 2 SMs, how are the blocks distributed across them?
- Blocks are assigned round-robin
 - Block 0 → SM 0
 - Block 1 → SM 1
 - Block 2 → SM 0
 - Block 3 → SM 1

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many warps does each SM execute?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How many warps does each SM execute?
- Warps per SM = Total Warps / SMs

$$= 16 / 2$$

$$= 8$$

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How does the GPU schedule warps within an SM?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- How does the GPU schedule warps within an SM?
- Round-robin execution: The SM cycles through available warps, executing one per cycle if resources allow.

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<<gridDim, blockDim>>>();
```

- At the end of execution, how many threads have been scheduled?

Thread Scheduling Example

```
dim3 blockDim(128);  
dim3 gridDim(4);  
kernel<<gridDim, blockDim>>>();
```

- At the end of execution, how many threads have been scheduled?
- All 512 threads (since every warp executes all its 32 threads).

References

- David Kirk and John Nickolls, “Graphics and Computing GPUs”, in *Computer Organization and Design: The Hardware/software Interface*, 2012.



Heading

- Text